

DOM 조작과 이벤트 처리

학습 목표 - DOM 트리와 노드 구조를 이해하고 설명할 수 있습니다 - 요소를 선택·생성·삭제하여 페이지를 동적으로 변경할 수 있습니다 - addEventListener와 이벤트 객체로 사용자 상호작용을 처리할 수 있습니다 - 캡처링·버블링을 이해하고 이벤트 위임으로 효율적인 UI를 구현할 수 있습니다

DOM 트리 이해하기

도입

지금까지 HTML로 구조를 만들고 CSS로 꾸몄습니다. 이제 JavaScript가 무대에 오를 차례입니다. JavaScript가 웹 페이지를 "조작"한다고 할 때, 실제로는 **DOM(Document Object Model)**이라는 중간 계층을 통해 작업합니다. DOM이 무엇인지 이해하면, 버튼 클릭으로 내용을 바꾸거나 새 항목을 추가하는 모든 동적 기능의 원리가 명확해집니다.

핵심 개념 설명

DOM(Document Object Model)이란

DOM(Document Object Model, 문서 객체 모델)은 브라우저가 HTML 문서를 읽어 들인 뒤 JavaScript가 다룰 수 있도록 변환한 **객체 트리(tree of objects)**입니다.

브라우저가 HTML 파일을 해석하면, 단순한 텍스트 문자열이 아닌 서로 연결된 **노드(Node)**들의 집합체로 바뀝니다. JavaScript는 이 노드들을 읽고, 수정하고, 추가하고, 삭제할 수 있습니다.

비유: 가게도처럼 부모-자식 관계로 연결된 가족 나무를 생각해 보십시오. html이 뿌리이고, 그 아래 head와 body가 가지를 치며, 각 가지에서 또 자식 요소들이 뻗어 나옵니다. JavaScript는 이 가게도를 보면서 원하는 가족 구성원을 찾아 정보를 변경할 수 있습니다.

중요: DOM과 HTML 소스 코드는 같지 않습니다. JavaScript로 DOM을 변경하면 화면은 바뀌지만 원본 `.html` 파일은 그대로입니다. 브라우저 개발자 도구의 "Elements" 탭에서 보이는 것이 바로 현재 DOM 상태입니다.

노드의 종류

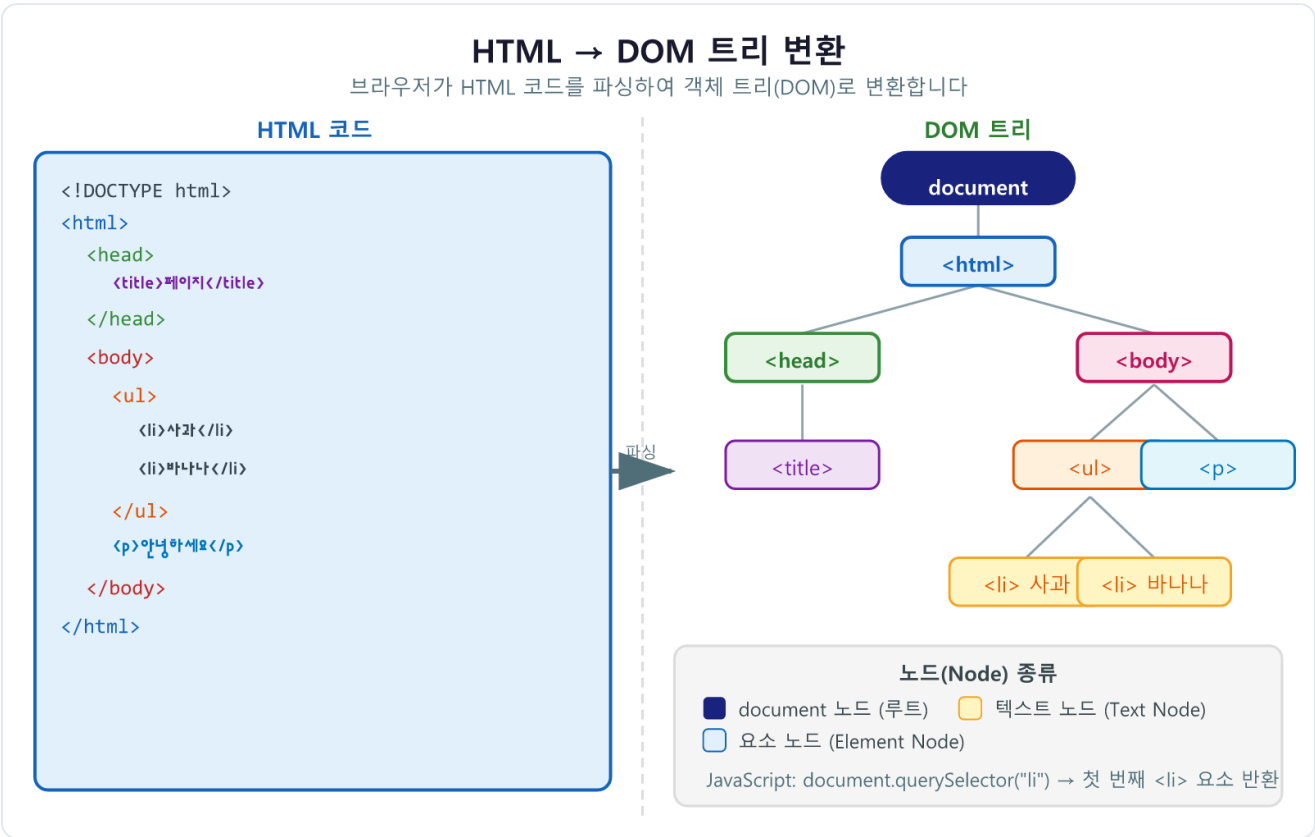
DOM 트리를 구성하는 **노드(Node)**에는 세 가지 주요 종류가 있습니다.

노드 종류	설명	예시
요소 노드(Element Node)	HTML 태그 하나하나	<code><p></code> , <code></code> , <code><div></code>
텍스트 노드(Text Node)	태그 사이의 텍스트	"안녕하세요"
속성 노드(Attribute Node)	요소의 속성	<code>class="btn"</code> , <code>href="/"</code>

가장 많이 다루는 것은 **요소 노드**이며, 속성은 요소 노드의 일부로 취급합니다.

HTML과 DOM 트리의 관계

다음 HTML을 보면서 DOM 트리가 어떻게 형성되는지 살펴봅시다.



HTML→DOM 트리 변환

코드로 확인하기

`document` 객체는 DOM의 진입점(entry point)입니다. `document` 를 통해 페이지의 모든 요소에 접근할 수 있습니다.

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>DOM 탐색 실습</title>
</head>
<body>
  <ul id="fruits">
    <li class="item">사과</li>
    <li class="item">바나나</li>
    <li class="item">포도</li>
  </ul>

  <script>
    // document가 DOM의 최상위 객체입니다
    console.log(document.title);           // 콘솔 출력: DOM 탐색 실습

    // body 요소 자체에 접근합니다
    console.log(document.body);           // 콘솔 출력: <body> ... </body>

    // id로 요소를 찾습니다
    const list = document.getElementById('fruits');
    console.log(list);                     // 콘솔 출력: <ul id="fruits"> ... </ul>

    // 자식 요소들의 개수를 확인합니다
    console.log(list.children.length);     // 콘솔 출력: 3

    // 첫 번째 자식 요소를 확인합니다
    console.log(list.firstChild.textContent); // 콘솔 출력: 사과
  </script>
</body>
</html>
```

브라우저 콘솔 출력:

DOM 탐색 실습

```
<body> ... </body>
```

```
<ul id="fruits"> ... </ul>
```

3

사과

줄별 코드 설명

줄	코드	설명
18	<code>console.log(document.title)</code>	<code>document.title</code> 은 <code><title></code> 태그의 텍스트를 반환합니다
21	<code>console.log(document.body)</code>	<code>document.body</code> 는 <code><body></code> 요소 노드 자체를 반환합니다
24	<code>document.getElementById('fruit...</code>	<code>id</code> 속성이 <code>"fruits"</code> 인 요소를 찾습니다
27	<code>list.children.length</code>	<code>children</code> 은 자식 요소 노드 만 모은 컬렉션입니다(텍스트 노드 제외)
30	<code>list.firstElementChild.textContent...</code>	첫 번째 자식 요소의 텍스트 내용을 읽습니다

절 요약

- DOM은 브라우저가 HTML을 파싱해 만드는 **객체 트리**이며, JavaScript의 조작 대상입니다
- 트리의 각 구성원을 **노드**라 하고, 요소·텍스트·속성 노드로 나눕니다
- `document` 객체가 DOM의 최상위 진입점이며 모든 탐색은 여기서 시작합니다

요소 선택하고 조작하기

도입

DOM 트리가 무엇인지 알았다면, 이제 원하는 요소를 "찾아" 내용을 바꾸는 작업을 배울 차례입니다. 이 절에서는 현대 JavaScript에서 표준으로 사용되는 `querySelector` 와 `querySelectorAll` 을 중심으로, 텍스트 변경·스타일 변경·클래스 조작까지 모두 다룹니다. 특히 `innerHTML` 과 `textContent` 의 보안 차이는 반드시 짚어야 할 핵심 내용입니다.

핵심 개념 설명

querySelector / querySelectorAll

document.querySelector(selector)

항목	내용
설명	CSS 선택자와 일치하는 첫 번째 요소를 반환합니다
매개변수	selector (string, 필수): CSS 선택자 문자열
반환값	Element - 일치하는 첫 번째 요소, 없으면 null
예외	SyntaxError - 선택자 문법이 잘못된 경우
사용 예시	<code>document.querySelector('.btn')</code> → 클래스가 btn 인 첫 번째 요소

document.querySelectorAll(selector)

항목	내용
설명	CSS 선택자와 일치하는 모든 요소를 NodeList 로 반환합니다
매개변수	selector (string, 필수): CSS 선택자 문자열
반환값	NodeList - 일치하는 모든 요소의 정적 컬렉션
사용 예시	<code>document.querySelectorAll('li')</code> → 모든 요소 목록

textContent와 innerHTML의 차이 — 보안 주의

요소의 내용을 바꿀 때 **textContent** 와 **innerHTML** 중 무엇을 써야 할지 반드시 알아야 합니다.

속성	설명	보안
textContent	텍스트만 처리. HTML 태그를 문자 그대로 표시	안전
innerHTML	HTML을 파싱하여 실제 요소로 삽입	주의 필요

보안 경고(XSS 취약점): `innerHTML` 에 사용자 입력을 직접 대입하면 **XSS(Cross-Site Scripting, 교차 사이트 스크립팅)** 공격에 노출됩니다. 예를 들어 사용자가 `<img src='x' onerror='악성코드()>` 같은 문자열을 입력하면 그 코드가 실행됩니다. **사용자 입력 값은 반드시 `textContent` 로 처리하십시오.**

classList로 클래스 조작하기

요소의 CSS 클래스를 JavaScript로 추가·제거·토글하는 가장 안전하고 권장되는 방법은 `classList` API입니다.

`element.classList`

항목	내용
설명	요소의 CSS 클래스 목록을 관리하는 <code>DOMTokenList</code> 객체
주요 메서드	<code>add('cls')</code> — 클래스 추가
	<code>remove('cls')</code> — 클래스 제거
	<code>toggle('cls')</code> — 있으면 제거, 없으면 추가
	<code>contains('cls')</code> — 포함 여부 <code>boolean</code> 반환
사용 예시	<code>btn.classList.toggle('active')</code>

코드로 확인하기

다음 코드는 `querySelector` 로 요소를 선택하고, `textContent` 와 `classList` 로 내용과 스타일을 동적으로 변경하는 예제입니다.

```

<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>요소 조작 예제</title>
  <style>
    .highlight { background-color: #fff3cd; font-weight: bold; }
    .completed { text-decoration: line-through; color: #999; }
    #status { margin-top: 12px; font-size: 0.9rem; color: #555; }
  </style>
</head>
<body>
  <h2 id="heading">오늘의 할 일</h2>
  <ul id="task-list">
    <li id="task-1">장보기</li>
    <li id="task-2">운동하기</li>
    <li id="task-3">독서하기</li>
  </ul>
  <button id="btn-complete">첫 번째 완료 처리</button>
  <button id="btn-highlight">제목 강조 토글</button>
  <p id="status"></p>

  <script>
    // 요소를 선택합니다
    const heading = document.querySelector('#heading');
    const firstTask = document.querySelector('#task-1');
    const allTasks = document.querySelectorAll('#task-list li');
    const statusEl = document.querySelector('#status');

    // textContent로 안전하게 텍스트를 변경합니다
    statusEl.textContent = `총 ${allTasks.length}개의 할 일이 있습니다.`;

    // classList로 클래스를 추가합니다
    document.querySelector('#btn-complete').addEventListener('click', () => {
      firstTask.classList.add('completed');
      firstTask.textContent = '장보기 (완료)';
    });

    // classList.toggle로 클래스를 켜고 끕니다
    document.querySelector('#btn-highlight').addEventListener('click', () => {
      heading.classList.toggle('highlight');
    });

    // 모든 li 요소를 순회합니다
    allTasks.forEach((task, index) => {
      console.log(`${index + 1}번 작업: ${task.textContent}`);
    });
  </script>

```

```
</body>
</html>
```

브라우저 동작 결과: - 페이지 로드 시 하단에 "총 3개의 할 일이 있습니다." 텍스트가 표시됩니다 - "첫 번째 완료 처리" 버튼 클릭 → "장보기" 항목에 취소선이 생기고 회색으로 바뀝니다 - "제목 강조 토글" 버튼 클릭 → 제목 배경이 노란색으로 바뀌고 다시 클릭하면 원래대로 돌아옵니다

콘솔 출력:

```
1번 작업 : 장보기
2번 작업 : 운동하기
3번 작업 : 독서하기
```

줄별 코드 설명

줄	코드	설명
22	<code>document.querySelector('#headi...</code>	<code>#heading</code> 은 id 선택자입니다. CSS 선택자 문법을 그대로 사용합니다
24	<code>document.querySelectorAll('#ta...</code>	자손 결합자로 <code>#task-list</code> 안의 모든 <code>li</code> 를 선택합니다
28	<code>statusEl.textContent = ...</code>	<code>textContent</code> 는 HTML을 파싱하지 않고 순수 텍스트로만 삽입하므로 안전합니다
31	<code>firstTask.classList.add('compl...</code>	<code>completed</code> 클래스를 추가합니다. CSS에서 정의한 취소선 스타일이 적용됩니다
37	<code>heading.classList.toggle('high...</code>	<code>highlight</code> 클래스가 없으면 추가하고, 있으면 제거합니다
42	<code>allTasks.forEach(...)</code>	<code>NodeList</code> 는 <code>forEach</code> 를 직접 사용할 수 있습니다

팁: `querySelectorAll` 이 반환하는 `NodeList` 는 배열처럼 `forEach` 를 사용할 수 있습니다. 하지만 `map` , `filter` 같은 배열 메서드는 기본적으로 없습니다. 배열로 변환하려면 `Array.from(nodeList)` 또는 `[...nodeList]` 를 사용하십시오.

베스트 프랙티스: `querySelector` / `querySelectorAll` 을 사용하면 CSS 선택자를 그대로 쓸 수 있어 직관적입니다. `getElementById` 는 속도가 약간 빠르지만, `querySelector('#id')` 로 대체해도 실무에서 체감 차이는 없습니다.

속성 읽고 쓰기

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>속성 조작 예제</title>
</head>
<body>
  <a id="link" href="https://example.com" target="_blank">예시 링크</a>
  

  <script>
    const link = document.querySelector('#link');
    const img = document.querySelector('#avatar');

    // getAttribute로 속성 값을 읽습니다
    console.log(link.getAttribute('href')); // 콘솔 출력: https://example.com

    // setAttribute로 속성 값을 변경합니다
    img.setAttribute('src', 'profile.png');
    img.setAttribute('alt', '사용자 프로필 사진');

    // 자주 쓰는 속성은 직접 프로퍼티로도 접근할 수 있습니다
    console.log(link.href); // 콘솔 출력: https://example.com/
    console.log(img.alt); // 콘솔 출력: 사용자 프로필 사진
  </script>
</body>
</html>
```

줄별 코드 설명

줄	코드	설명
14	<code>link.getAttribute('href')</code>	속성의 원본 문자열 값을 반환합니다
17	<code>img.setAttribute('src', 'profi...</code>	<code>src</code> 속성을 새 값으로 변경합니다. 이미지가 즉시 교체됩니다
21	<code>link.href</code>	DOM 프로퍼티로 직접 접근합니다. <code>getAttribute</code> 보다 간결합니다

절 요약

- `querySelector` / `querySelectorAll` 로 CSS 선택자를 이용해 요소를 선택합니다
- 텍스트 변경에는 **textContent**를, 사용자 입력 표시에 `innerHTML` 사용은 **피해야** 합니다
- `classList.add/remove/toggle` 로 클래스를 동적으로 조작하면 CSS와 JavaScript를 깔끔히 분리할 수 있습니다

요소 생성과 삭제

도입

지금까지는 이미 HTML에 있는 요소를 선택해 변경했습니다. 이제 한 단계 더 나아가, JavaScript만으로 새 요소를 **만들어** DOM 트리에 추가하거나 기존 요소를 **제거**하는 방법을 배웁니다. 이 기능이 있어야 할 일을 입력받아 목록에 추가하거나, 댓글을 동적으로 렌더링하는 웹 앱을 만들 수 있습니다.

핵심 개념 설명

createElement와 append

```
document.createElement(tagName)
```

항목	내용
설명	지정한 태그 이름의 새 요소를 메모리에 생성합니다 (아직 DOM에 추가되지 않은 상태)
매개변수	<code>tagName</code> (string, 필수): 생성할 HTML 태그 이름
반환값	<code>HTMLElement</code> - 새로 생성된 요소
사용 예시	<code>document.createElement('li')</code> → 새 <code></code> 요소

`element.append( ... nodes )`

항목	내용
설명	요소의 마지막 자식으로 노드나 문자열을 추가합니다
매개변수	<code>nodes</code> (Node 또는 string, 복수 가능): 추가할 노드 또는 텍스트
반환값	<code>undefined</code>
사용 예시	<code>ul.append(li)</code> — <code>ul</code> 의 마지막에 <code>li</code> 추가

`appendChild` vs `append` : `appendChild` 는 단일 노드만 추가하는 구형 API입니다. `append` 는 여러 노드와 문자열을 한 번에 추가할 수 있는 모던 API입니다. 특별한 이유가 없다면 `append` 를 사용하십시오.

DocumentFragment로 성능 최적화

여러 요소를 한꺼번에 추가할 때 반복문에서 매번 DOM에 직접 추가하면, 추가할 때마다 브라우저가 화면을 다시 계산(리플로우)합니다. **DocumentFragment(문서 단편)**를 사용하면 메모리 안에서 요소들을 조립한 뒤 DOM에 한 번에 추가할 수 있습니다.

코드로 확인하기

다음 코드는 배열 데이터를 받아 동적으로 목록 항목을 생성하고 DOM에 추가합니다.

```

<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>요소 생성과 삭제</title>
  <style>
    #book-list { list-style: none; padding: 0; }
    .book-item {
      display: flex;
      justify-content: space-between;
      align-items: center;
      padding: 8px 12px;
      border: 1px solid #ddd;
      border-radius: 4px;
      margin-bottom: 6px;
    }
    .delete-btn {
      background: #e74c3c;
      color: white;
      border: none;
      padding: 4px 10px;
      border-radius: 4px;
      cursor: pointer;
    }
  </style>
</head>
<body>
  <h2>내 도서 목록</h2>
  <ul id="book-list"></ul>

  <script>
    const books = ['자바스크립트 완벽 가이드', '클린 코드', '리팩터링'];
    const bookList = document.querySelector('#book-list');

    // DocumentFragment를 사용하여 성능을 최적화합니다
    const fragment = document.createDocumentFragment();

    books.forEach((title) => {
      // 새 li 요소를 생성합니다
      const li = document.createElement('li');
      li.classList.add('book-item');

      // 책 제목 span을 생성합니다 (textContent로 안전하게 삽입)
      const titleSpan = document.createElement('span');
      titleSpan.textContent = title;

      // 삭제 버튼을 생성합니다
      const deleteBtn = document.createElement('button');
      deleteBtn.classList.add('delete-btn');
      deleteBtn.textContent = '삭제';

```

```

// 삭제 버튼 클릭 시 해당 li를 제거합니다
deleteBtn.addEventListener('click', () => {
  li.remove();
});

// 자식 요소들을 li에 추가합니다
li.append(titleSpan, deleteBtn);

// 아직 DOM에 추가하지 않고 fragment에 쌓습니다
fragment.append(li);
});

// 한 번에 DOM에 추가합니다 (리플로우 1회)
bookList.append(fragment);

console.log(`총 ${bookList.children.length}권 렌더링 완료`);
// 콘솔 출력: 총 3권 렌더링 완료
</script>
</body>
</html>

```

브라우저 동작 결과: - 페이지 로드 시 책 3권이 목록으로 표시되고, 각 항목 오른쪽에 "삭제" 버튼이 있습니다 - "삭제" 버튼 클릭 → 해당 항목이 목록에서 즉시 제거됩니다

줄별 코드 설명

줄	코드	설명
31	<code>document.createDocumentFragmen...</code>	메모리 상의 가상 컨테이너를 만듭니다. DOM에 붙어있지 않아 리플로우가 발생하지 않습니다
36	<code>document.createElement('li')</code>	새 <code></code> 요소를 메모리에 생성합니다. 아직 화면에는 보이지 않습니다
40	<code>titleSpan.textContent = title</code>	<code>innerHTML</code> 대신 <code>textContent</code> 를 사용해 XSS를 방지합니다
50	<code>li.remove()</code>	요소 자신을 DOM에서 직접 제거하는 간결한 메서드입니다
54	<code>li.append(titleSpan, deleteBtn)</code>	두 자식 요소를 한 번에 추가합니다
57	<code>bookList.append(fragment)</code>	미리 조립한 fragment를 DOM에 한 번에 붙입니다. 리플로우가 1회만 발생합니다

베스트 프랙티스: 요소를 동적으로 생성할 때 텍스트는 항상 `textContent` 로 삽입하십시오. `innerHTML` 로 직접 문자열을 조합하면 코드가 간결해 보이지만 XSS 위험이 생깁니다. `createElement` + `textContent` 의 조합이 안전하고 올바른 방법입니다.

절 요약

- `createElement` 로 요소를 생성하고 `append` 로 DOM에 추가합니다
- `element.remove()` 로 요소 자신을 DOM에서 삭제합니다
- 여러 요소를 한꺼번에 추가할 때는 **DocumentFragment**를 사용해 불필요한 리플로우를 줄입니다

이벤트와 이벤트 흐름

도입

버튼 클릭, 키보드 입력, 마우스 이동... 사용자가 페이지와 상호작용할 때마다 브라우저는 **이벤트(Event)**를 발생시킵니다. JavaScript는 이 이벤트를 감지해 원하는 동작을 실행합니다. 이 절에서는 이벤트의 기본 처리 방법과 함께, 많은 개발자가 처음에 놓치는 **이벤트 버블링(Bubbling)**과 **캡처링(Capturing)** 개념을 명확히 이해합니다.

핵심 개념 설명

`addEventListener`

```
element.addEventListener(type, listener, options)
```

항목	내용
설명	요소에 이벤트 리스너를 등록합니다
매개변수	<code>type</code> (string, 필수): 이벤트 유형 (<code>'click'</code> , <code>'input'</code> , <code>'keydown'</code> 등)
	<code>listener</code> (Function, 필수): 이벤트 발생 시 호출될 콜백 함수
	<code>options</code> (object 또는 boolean, 선택): <code>{ capture: false, once: false, passive: false }</code>
반환값	<code>undefined</code>
사용 예시	<code>btn.addEventListener('click', handleClick)</code>

주의: HTML 속성으로 이벤트를 달던 `onclick="myFunc()"` 방식은 관심사 분리 원칙에 어긋나며 유지보수가 어렵습니다. 항상 `addEventListener` 를 사용하십시오.

이벤트 흐름: 캡처링 → 타깃 → 버블링

이벤트가 발생하면 단순히 클릭된 요소에서만 처리되는 것이 아닙니다. DOM 트리를 따라 **3 단계**로 전파됩니다.

이벤트 흐름: 캡처링·타깃·버블링

단계	방향	설명
1단계: 캡처링 (Capturing)	document → 타깃	이벤트가 위에서 아래로 내려가며 조상 요소들을 통과합니다
2단계: 타깃(Target)	타깃 요소	실제로 이벤트가 발생한 요소에 도달합니다
3단계: 버블링 (Bubbling)	타깃 → document	이벤트가 아래에서 위로 올라가며 조상 요소들로 전파됩니다

비유: 수면에 돌을 던지면 물방울이 깊은 곳까지 내려갔다가(캡처링) 다시 수면 위로

떠오르는(버블링) 과정과 같습니다.

기본적으로 `addEventListener` 는 **버블링 단계**에서 이벤트를 처리합니다. 세 번째 인수에 `true` 를 주면 캡처링 단계에서 처리합니다.

`event.target`과 `event.currentTarget`의 차이

속성	의미
<code>event.target</code>	이벤트가 실제로 발생한 요소 (버블링 중에도 변하지 않음)
<code>event.currentTarget</code>	현재 이벤트 핸들러가 등록된 요소 (버블링에 따라 변함)

`preventDefault`와 `stopPropagation`

`event.preventDefault()`

항목	내용
설명	이벤트의 기본 동작을 취소합니다 (전파는 계속 됨)
사용 예시	링크 클릭 시 페이지 이동 막기, 폼 제출 막기

`event.stopPropagation()`

항목	내용
설명	이벤트의 전파(버블링/캡처링)를 중단합니다 (기본 동작은 계속됨)
사용 예시	모달 내부 클릭이 배경 클릭으로 인식되지 않게 막기

코드로 확인하기

버블링 확인 예제

다음 코드는 버블링의 동작을 직접 관찰할 수 있는 예제입니다.

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>이벤트 버블링 확인</title>
  <style>
    .box {
      padding: 24px;
      border: 2px solid #3498db;
      border-radius: 6px;
      background: #ebf5fb;
      cursor: pointer;
    }
    #inner {
      padding: 16px;
      background: #3498db;
      color: white;
      border-radius: 4px;
      text-align: center;
    }
    #log { margin-top: 12px; font-size: 0.85rem; color: #333; }
  </style>
</head>
<body>
  <div id="outer" class="box">
    외부 div (outer)
    <div id="inner">내부 div (inner) - 여기를 클릭하세요</div>
  </div>
  <p id="log"></p>

  <script>
    const log = document.querySelector('#log');

    // outer 요소에 클릭 리스너를 등록합니다 (버블링 단계, 기본값)
    document.querySelector('#outer').addEventListener('click', (event) => {
      log.textContent += `[outer 처리] target: ${event.target.id}, currentTarget: ${event.currentTarget.id}\n`;
    });

    // inner 요소에 클릭 리스너를 등록합니다
    document.querySelector('#inner').addEventListener('click', (event) => {
      log.textContent += `[inner 처리] target: ${event.target.id}, currentTarget: ${event.currentTarget.id}\n`;
    });
  </script>
</body>
</html>
```

브라우저 동작 결과 (inner 클릭 시): - inner 리스너가 먼저 실행되고, 이어서 outer 리스너도 실행됩니다 — 버블링 - 두 경우 모두 `event.target` 은 `inner` 입니다 -
`event.currentTarget` 은 각 핸들러가 등록된 요소(`inner` , `outer`)로 다릅니다

[inner 처리] target: inner, currentTarget: inner | [outer 처리] target: inner,

줄별 코드 설명

줄	코드	설명
33	<code>addEventListener('click', (eve...</code>	이벤트 콜백에 전달되는 <code>event</code> 객체에 이벤트 정보가 담겨 있습니다
34	<code>event.target.id</code>	실제 클릭된 요소의 <code>id</code> 를 읽습니다. 항상 <code>inner</code> 입니다
34	<code>event.currentTarget.id</code>	현재 리스너가 등록된 요소의 <code>id</code> 를 읽습니다. 각 핸들러에서 다릅니다

stopPropagation으로 버블링 막기

```

<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>stopPropagation 예제</title>
  <style>
    #modal-backdrop {
      position: fixed; inset: 0;
      background: rgba(0,0,0,0.5);
      display: flex;
      align-items: center;
      justify-content: center;
    }
    #modal-box {
      background: white;
      padding: 32px;
      border-radius: 8px;
      min-width: 280px;
    }
    .hidden { display: none !important; }
  </style>
</head>
<body>
  <button id="open-btn">모달 열기</button>

  <div id="modal-backdrop" class="hidden">
    <div id="modal-box">
      <h3>안내</h3>
      <p>배경을 클릭하면 모달이 닫힙니다.</p>
      <button id="close-btn">닫기</button>
    </div>
  </div>

  <script>
    const backdrop = document.querySelector('#modal-backdrop');
    const modalBox = document.querySelector('#modal-box');

    // 모달 열기
    document.querySelector('#open-btn').addEventListener('click', () => {
      backdrop.classList.remove('hidden');
    });

    // 배경(backdrop) 클릭 시 모달을 닫습니다
    backdrop.addEventListener('click', () => {
      backdrop.classList.add('hidden');
    });

    // 모달 내부 클릭은 backdrop으로 버블링되지 않도록 막습니다
    modalBox.addEventListener('click', (event) => {
      event.stopPropagation();
    });
  </script>

```

```
});

// 닫기 버튼 클릭 시 모달을 닫습니다
document.querySelector('#close-btn').addEventListener('click', () => {
  backdrop.classList.add('hidden');
});
</script>
</body>
</html>
```

브라우저 동작 결과: - "모달 열기" 버튼 클릭 → 반투명 배경과 함께 모달이 나타납니다 - 모달 내부 클릭 → `stopPropagation` 으로 backdrop에 버블링이 막혀 모달이 닫히지 않습니다 - 모달 외부(배경) 클릭 → backdrop 리스너가 실행되어 모달이 닫힙니다

줄별 코드 설명

줄	코드	설명
41	<code>backdrop.addEventListener('cli...</code>	backdrop 클릭 시 모달을 닫는 리스너입니다
46	<code>event.stopPropagation()</code>	모달 박스 내부 클릭이 backdrop으로 버블링 되는 것을 막습니다
51	<code>backdrop.classList.add('hidden...</code>	<code>hidden</code> 클래스를 추가하면 CSS의 <code>display: none</code> 이 적용됩니다

주의: `stopPropagation` 은 꼭 필요한 경우에만 사용하십시오. 무분별하게 사용하면 다른 리스너가 이벤트를 수신하지 못해 예상치 못한 버그가 생깁니다.

절 요약

- `addEventListener` 로 이벤트를 등록하고, 콜백의 `event` 객체에서 정보를 읽습니다
- 이벤트는 **캡처링(하향)** → **타깃** → **버블링(상향)** 3단계로 전파됩니다
- `event.target` 은 실제 이벤트 발생 요소, `event.currentTarget` 은 리스너 등록 요소입니다
- `preventDefault` 는 기본 동작을, `stopPropagation` 은 전파를 막습니다

이벤트 위임과 동적 UI

도입

지금까지의 예제들은 미리 만들어진 요소에 리스너를 등록했습니다. 그런데 목록 항목이 동적으로 추가된다면 어떨까요? 새로 생성된 요소에는 이전에 등록한 리스너가 없어서 이벤트가 동작하지 않습니다. 이 문제를 우아하게 해결하는 기법이 **이벤트 위임(Event Delegation)**입니다.

핵심 개념 설명

이벤트 위임(Event Delegation)

이벤트 위임은 자식 요소 각각에 리스너를 달지 않고, **공통 부모 요소에 하나의 리스너를 달아** 버블링으로 모든 자식의 이벤트를 처리하는 기법입니다.

비유: 각 방마다 경비원을 두는 대신 건물 입구에 한 명을 두어 들어오는 사람을 확인하는 것과 같습니다. 새로운 방이 생겨도 입구 경비원은 그대로 유지됩니다.

이벤트 위임이 필요한 경우: 1. **동적으로 추가되는 요소**에 이벤트를 걸어야 할 때 2. **많은 수의 유사한 요소**에 같은 이벤트를 걸어야 할 때 (메모리 절약)

`event.target` 과 `Element.matches(selector)` 조합으로 실제로 클릭된 요소가 원하는 요소인지 확인합니다.

`element.matches(selector)`

항목	내용
설명	요소가 CSS 선택자와 일치하면 <code>true</code> , 아니면 <code>false</code> 를 반환합니다
매개변수	<code>selector</code> (string, 필수): CSS 선택자
반환값	<code>boolean</code>
사용 예시	<code>event.target.matches('.delete-btn')</code>

상태와 UI 동기화

동적 UI를 올바르게 만들려면 **상태(state)**와 **UI**를 명확히 구분해야 합니다. 상태는 JavaScript 변수에 저장되고, UI는 상태를 보고 그려집니다. 상태가 바뀌면 UI를 다시 그리는 패턴이 유지 보수하기 좋습니다.

코드로 확인하기

이벤트 위임 기본 패턴

```

<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>이벤트 위임 기본</title>
  <style>
    #color-list { list-style: none; padding: 0; display: flex; gap: 8px; flex-
    .color-chip {
      padding: 8px 16px;
      border-radius: 20px;
      cursor: pointer;
      border: 2px solid transparent;
      transition: border-color 0.2s;
    }
    .color-chip.selected { border-color: #2c3e50; }
    #selected-color { margin-top: 12px; font-weight: bold; }
  </style>
</head>
<body>
  <h3>색상 선택</h3>
  <ul id="color-list">
    <li class="color-chip" data-color="빨강" style="background:#e74c3c; color:v
    <li class="color-chip" data-color="파랑" style="background:#3498db; color:v
    <li class="color-chip" data-color="초록" style="background:#2ecc71; color:v
    <li class="color-chip" data-color="노랑" style="background:#f1c40f;">노랑</
  </ul>
  <p id="selected-color">선택된 색상: 없음</p>

  <script>
    const colorList = document.querySelector('#color-list');
    const selectedText = document.querySelector('#selected-color');

    // 부모 ul에 리스너를 한 번만 등록합니다 (이벤트 위임)
    colorList.addEventListener('click', (event) => {
      // event.target이 .color-chip 클래스를 가진 요소인지 확인합니다
      if (!event.target.matches('.color-chip')) return;

      // 기존 선택을 해제합니다
      colorList.querySelectorAll('.color-chip').forEach((chip) => {
        chip.classList.remove('selected');
      });

      // 클릭된 요소를 선택 상태로 만듭니다
      event.target.classList.add('selected');

      // data-* 속성에서 값을 읽어 UI를 업데이트합니다
      const color = event.target.dataset.color;
      selectedText.textContent = `선택된 색상: ${color}`;
    });
  </script>

```

```
</body>
</html>
```

브라우저 동작 결과: - 색상 칩 클릭 시 해당 칩에 테두리가 생기고 하단 텍스트가 선택된 색상 이름으로 바뀝니다 - 다른 칩을 클릭하면 이전 선택이 해제되고 새 칩이 선택됩니다

줄별 코드 설명

줄	코드	설명
28	<code>colorList.addEventListener('cl...</code>	자식 <code>li</code> 각각이 아닌 부모 <code>ul</code> 에 리스너를 등록합니다. 이것이 위임입니다
30	<code>if (!event.target.matches('.co...</code>	클릭이 <code>li</code> 안의 다른 요소에서 발생할 수 있으므로, 정확한 대상을 확인합니다
38	<code>event.target.dataset.color</code>	HTML의 <code>data-color</code> 속성을 <code>dataset.color</code> 로 읽습니다 (<code>data-</code> 접두사 제거, camelCase)

완전한 동적 UI: 이벤트 위임 + 상태 관리

다음 예제는 이벤트 위임을 활용해 동적으로 추가되는 항목의 이벤트도 한 번의 리스너로 처리합니다.


```

<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>동적 목록 with 이벤트 위임</title>
  <style>
    body { font-family: sans-serif; max-width: 480px; margin: 40px auto; padding: 0 16px; }
    #input-area { display: flex; gap: 8px; margin-bottom: 16px; }
    #new-item { flex: 1; padding: 8px; border: 1px solid #ccc; border-radius: 4px; }
    #add-btn { padding: 8px 16px; background: #3498db; color: white; border: none; }
    #item-list { list-style: none; padding: 0; }
    .list-item {
      display: flex;
      align-items: center;
      gap: 10px;
      padding: 10px 12px;
      border: 1px solid #eee;
      border-radius: 4px;
      margin-bottom: 6px;
    }
    .list-item.done .item-text { text-decoration: line-through; color: #aaa; }
    .item-text { flex: 1; }
    .done-btn { background: #2ecc71; color: white; border: none; padding: 4px 8px; }
    .del-btn { background: #e74c3c; color: white; border: none; padding: 4px 8px; }
    #count { font-size: 0.85rem; color: #777; margin-top: 8px; }
  </style>
</head>
<body>
  <h2>할 일 목록</h2>
  <div id="input-area">
    <input type="text" id="new-item" placeholder="새 할 일을 입력하세요">
    <button id="add-btn">추가</button>
  </div>
  <ul id="item-list"></ul>
  <p id="count"></p>

  <script>
    // — 상태 —————
    let items = [];
    let nextId = 1;

    // — DOM 참조 —————
    const input = document.querySelector('#new-item');
    const addBtn = document.querySelector('#add-btn');
    const itemList = document.querySelector('#item-list');
    const countEl = document.querySelector('#count');

    // — UI 렌더링 함수 —————
    // 상태(items 배열)를 받아 DOM을 새로 그립니다
    const renderList = () => {

```

```

// 기존 목록을 비웁니다
itemList.innerHTML = '';

const fragment = document.createDocumentFragment();

items.forEach((item) => {
  const li = document.createElement('li');
  li.classList.add('list-item');
  if (item.done) li.classList.add('done');
  li.dataset.id = item.id; // 식별자를 data 속성에 저장합니다

  const span = document.createElement('span');
  span.classList.add('item-text');
  span.textContent = item.text; // textContent로 안전하게 삽입합니다

  const doneBtn = document.createElement('button');
  doneBtn.classList.add('done-btn');
  doneBtn.textContent = item.done ? '취소' : '완료';
  doneBtn.dataset.action = 'done';

  const delBtn = document.createElement('button');
  delBtn.classList.add('del-btn');
  delBtn.textContent = '삭제';
  delBtn.dataset.action = 'delete';

  li.append(span, doneBtn, delBtn);
  fragment.append(li);
});

itemList.append(fragment);

const remaining = items.filter((i) => !i.done).length;
countEl.textContent = `전체 ${items.length}개 | 남은 항목 ${remaining}개`;
};

// — 이벤트 핸들러 —
// 항목 추가
const addItem = () => {
  const text = input.value.trim();
  if (!text) return;

  items.push({ id: nextId++, text, done: false });
  input.value = '';
  renderList(); // 상태가 바뀌면 UI를 다시 그립니다
};

addBtn.addEventListener('click', addItem);

// Enter 키로도 추가할 수 있게 합니다
input.addEventListener('keydown', (event) => {
  if (event.key === 'Enter') addItem();
});

```

```

// 이벤트 위임: ul 하나에만 등록하여 동적으로 추가된 항목도 처리합니다
itemList.addEventListener('click', (event) => {
  // 클릭된 요소에서 data-action을 확인합니다
  const action = event.target.dataset.action;
  if (!action) return;

  // 가장 가까운 .list-item을 찾아 id를 읽습니다
  const li = event.target.closest('.list-item');
  const id = Number(li.dataset.id);

  if (action === 'done') {
    // 상태를 토글합니다
    items = items.map((item) =>
      item.id === id ? { ...item, done: !item.done } : item
    );
  } else if (action === 'delete') {
    // 배열에서 제거합니다
    items = items.filter((item) => item.id !== id);
  }

  renderList(); // 상태가 바뀌면 UI를 다시 그립니다
});

// 초기 렌더링
renderList();
</script>
</body>
</html>

```

브라우저 동작 결과: - 입력창에 텍스트를 입력하고 "추가" 버튼이나 Enter 키를 누르면 목록에 항목이 추가됩니다 - "완료" 버튼 클릭 → 항목에 취소선이 생기고 버튼 텍스트가 "취소"로 바뀝니다 - "삭제" 버튼 클릭 → 항목이 목록에서 제거되고 카운트가 업데이트됩니다 - 동적으로 추가된 항목도 완료/삭제가 정상 동작합니다 (이벤트 위임 덕분)

줄별 코드 설명

줄	코드	설명
43	<code>let items = []</code>	목록 데이터를 담는 상태 배열입니다. UI의 원천(source of truth)입니다
52	<code>const renderList = () => { ... }</code>	상태를 받아 DOM을 그리는 순수한 렌더링 함수입니다
54	<code>itemList.innerHTML = ''</code>	렌더링 전 기존 목록을 비웁니다. 여기서는 렌더링 함수 내부에서만 사용하므로 안전합니다
62	<code>li.dataset.id = item.id</code>	요소에 <code>data-id</code> 속성을 저장해 두면 이후 이벤트에서 어떤 항목인지 알 수 있습니다
88	<code>itemList.addEventListener('cli...</code>	<code>ul</code> 에 단 하나의 리스너로 모든 버튼 클릭 을 처리합니다. 이것이 이벤트 위임입니다
91	<code>event.target.closest('.list-it...</code>	클릭된 버튼에서 가장 가까운 조상 <code>.list-item</code> 을 찾습니다. 버블링 중 어느 위치에서도 안정적으로 동작합니다
96	<code>items.map((item) => ...)</code>	원본 배열을 직접 변경하지 않고 새 배열 을 반환합니다. 상태 불변성 패턴입니다
99	<code>items.filter(...)</code>	삭제할 항목을 제외한 새 배열을 만들어 <code>items</code> 를 교체합니다
103	<code>renderList()</code>	상태가 변경될 때마다 <code>renderList()</code> 를 호출하여 UI와 상태를 동기화합니다

베스트 프랙티스: `data-action` 패턴처럼 버튼마다 역할을 `data-*` 속성에 명시하면, 이벤트 위임 핸들러에서 어떤 동작을 해야 하는지 `if/switch`로 명확하게 분기할 수 있습니다.

팁: `element.closest(selector)`는 자기 자신을 포함한 조상들 중에서 선택자와 일치하는 **가장 가까운** 요소를 반환합니다. 이벤트 위임에서 "어느 항목에서 발생한 이벤트인가"를 찾을 때 매우 유용합니다.

절 요약

- 이벤트 위임은 **부모 요소에 리스너를 달고** `event.target`으로 실제 클릭 요소를 확인하는 기법입니다

- 동적으로 추가된 요소도 별도 리스너 없이 처리할 수 있어 메모리 효율적입니다
 - `data-*` 속성과 `element.closest()` 를 조합하면 위임 코드가 깔끔해집니다
 - 상태와 UI를 분리하여 **상태가 바뀌면 UI를 다시 그리는** 패턴을 일관되게 적용하십시오
-

FAQ

Q1: `querySelector` 와 `getElementById` 중 무엇을 써야 하나요?

A1: `getElementById` 는 id만 사용 가능한 반면, `querySelector` 는 CSS 선택자를 모두 지원합니다. 성능 차이는 현대 브라우저에서 거의 없으므로, 일관성을 위해 `querySelector('#id-name')` 형태를 사용하는 것이 좋습니다. 단, 반복 호출이 매우 많은 극한 성능 상황에서는 `getElementById` 가 약간 빠릅니다.

Q2: `innerHTML` 대신 `textContent`만 써야 하나요? `innerHTML`은 절대 사용하면 안 되나요?

A2: 무조건 금지는 아닙니다. **신뢰할 수 있는 HTML 문자열**(서버에서 받은 이미 검증된 HTML, 직접 작성한 리터럴 문자열)이라면 `innerHTML` 을 사용할 수 있습니다. 그러나 **사용자 입력값**을 그대로 `innerHTML` 에 대입하는 것은 XSS 위험이 있어 절대로 해서는 안 됩니다. 동적 요소 생성은 `createElement` + `textContent` 조합이 가장 안전합니다.

Q3: 이벤트 위임을 쓸 때 캡처링 단계의 이벤트도 처리할 수 있나요?

A3: 네. `addEventListener` 의 세 번째 인수에 `{ capture: true }` 를 전달하면 캡처링 단계에서 이벤트를 처리합니다. 다만 `focus` 나 `blur` 처럼 버블링되지 않는 이벤트를 위임으로 처리해야 할 때 주로 사용합니다(`focusin` / `focusout` 은 버블링됩니다). 일반적인 위임에는 기본값(버블링)을 사용하면 충분합니다.

Q4: `stopPropagation()` 과 `preventDefault()` 를 동시에 쓰고 싶을 때는 어떻게 하나요?

A4: 두 메서드를 모두 호출하면 됩니다.

```
element.addEventListener('click', (event) => {
  event.preventDefault();    // 기본 동작 취소 (예: 링크 이동)
  event.stopPropagation();   // 버블링 중단
});
```

또는 `event.stopImmediatePropagation()` 을 사용하면 같은 요소의 나머지 리스너들도 실행되지 않습니다.

Q5: `NodeList` 와 `Array` 의 차이는 무엇인가요?

A5: `querySelectorAll` 은 `NodeList` 를 반환합니다. `NodeList` 는 `forEach` 를 지원하지만 `map` , `filter` , `reduce` 같은 배열 메서드는 없습니다. 배열 메서드가 필요하다면 `Array.from(nodeList)` 또는 스프레드 연산자 `[...nodeList]` 로 배열로 변환한 뒤 사용하십시오.

장 요약

이번 장에서는 브라우저가 HTML을 **DOM 트리**로 변환한다는 원리를 이해하고, `querySelector` 와 `querySelectorAll` 로 요소를 선택하는 법을 배웠습니다. 텍스트를 변경할 때는 보안상 `textContent` 를 `innerHTML` 보다 우선 사용해야 함을 확인했습니다. `createElement` 와 `append` 로 요소를 동적으로 만들고 DOM에 추가하는 법, 그리고 `remove()` 로 삭제하는 법도 다뤘습니다. 이벤트의 캡처링·버블링 3단계 전파 원리를 이해하고, 버블링을 영리하게 활용한 **이벤트 위임** 기법으로 동적 UI를 효율적으로 구현하는 패턴까지 학습했습니다.

핵심 키워드

`DOM` , `querySelector` , `textContent` , `innerHTML(XSS 주의)` , `classList` , `createElement` , `append` , `addEventListener` , `event.target` , 버블링(Bubbling) , 캡처링(Capturing) , 이벤트 위임(Event Delegation) , `data-*` , `closest()`

다음 장 미리보기

다음 장에서는 **비동기 JavaScript**를 다룹니다. 이벤트 루프의 동작 원리를 이해하고, `Promise` 와 `async/await` 로 외부 API에서 데이터를 가져온 뒤 이번 장에서 배운 DOM 조작

으로 화면에 렌더링하는 완성도 높은 웹 앱을 만들어 봅니다.

✂ 실습 프로젝트 (Hands-on Project)

ch_11 실습 프로젝트: 투두 리스트(To-Do List)

이 장에서 배운 DOM 조작, 이벤트 처리, 이벤트 위임 개념을 실무 시나리오에 적용하는 프로젝트입니다.

초급: 기본 투두 리스트

시나리오

개인 일정 관리 앱의 핵심 기능인 투두 리스트를 만듭니다. 입력창에 할 일을 작성하고 추가하면 목록에 표시되고, 완료 체크와 삭제 기능이 동작하는 기본형 앱입니다. 사용자 입력은 보안을 위해 반드시 `textContent`로 처리합니다.

학습 목표

- `createElement` + `textContent` 로 동적 DOM 생성 패턴을 익힙니다
- `addEventListener` 로 버튼 클릭과 키보드 이벤트를 처리합니다
- `classList.toggle` 로 완료 상태를 시각적으로 반영합니다
- `element.remove()` 로 요소를 DOM에서 제거합니다

진행 방법

1. `project_starter.html` 을 열고 `// TODO` 주석을 찾습니다
2. 각 TODO를 이 장에서 배운 개념으로 완성합니다
3. 브라우저에서 열어 할 일을 추가/완료/삭제해보며 결과를 확인합니다

실행 방법

`project_starter.html` 파일을 브라우저에서 직접 열거나 Live Server로 실행하세요.

완성 기준

- [] 입력창에 텍스트 입력 후 "추가" 버튼 또는 Enter 키로 항목 추가
- [] 빈 입력은 추가되지 않습니다
- [] 각 항목에 "완료" 버튼 — 클릭 시 취소선 표시
- [] 각 항목에 "삭제" 버튼 — 클릭 시 목록에서 제거
- [] 남은 항목 수가 실시간으로 표시됩니다

중급: 필터 + 로컬 저장소 투두 리스트

시나리오

초급 투두 리스트를 확장하여 실제 앱처럼 동작하는 버전을 구현합니다. "전체 / 진행중 / 완료" 필터 탭으로 항목을 분류하고, 이벤트 위임으로 동적 항목을 처리하며, `localStorage` 로 새로고침 후에도 데이터가 유지됩니다.

학습 목표

- 이벤트 위임으로 동적 생성 항목의 이벤트를 효율적으로 처리합니다
- `data-*` 속성과 `closest()` 를 조합한 위임 패턴을 실전 적용합니다
- 상태(state)와 UI를 분리하여 `renderList()` 함수로 일관되게 렌더링합니다
- `localStorage` 의 `setItem` / `getItem` / `JSON.stringify` / `JSON.parse` 로 데이터를 영속화합니다

진행 방법

1. `project_advanced_starter.html` 을 열고 `// TODO` 주석을 찾습니다
2. 초급에서 연습한 내용을 바탕으로 더 복잡한 로직을 완성합니다
3. 브라우저에서 실행한 후 새로고침해도 데이터가 유지되는지 확인합니다

실행 방법

`project_advanced_starter.html` 파일을 브라우저에서 직접 열거나 Live Server로 실행

완성 기준

- [] 초급 기준 모두 충족
- [] 이벤트 위임으로 UI에 단 하나의 클릭 리스너만 사용

- [] "전체 / 진행중 / 완료" 필터 탭 동작
- [] 필터 탭도 이벤트 위임으로 처리
- [] 새로고침 후에도 localStorage로 데이터 복원
- [] "전체 삭제" 버튼으로 모든 항목 제거

확장 아이디어

- 항목을 드래그하여 순서를 변경하는 기능 (dragstart / dragover / drop 이벤트)
- 항목 텍스트를 더블클릭하면 인라인 편집 가능하게 변경
- 완료 항목만 일괄 삭제하는 "완료 항목 정리" 버튼 추가

▶  project_starter.html (스타터 코드 - 직접 완성해보세요)

▶  project_complete.html (완성 코드)

▶  project_advanced_starter.html (스타터 코드 - 직접 완성해보세요)

▶  project_advanced_complete.html (완성 코드)